

Propuesta de backend por fases

Cómo construir el lado servidor de la plataforma: qué se hace en cada fase, en qué orden, con qué decisiones técnicas y con qué criterio para dar cada fase por terminada.

Documento técnico · versión 1

Punto de partida: **frontend completo (21 rutas)**, **backend en 0 bytes**

Alcance estimado: **7 fases** · **~20 semanas** · primer entorno usable en la fase 3

— Contenido

1	Resumen ejecutivo	F6	Operación del activo
2	Punto de partida	F7	Endurecimiento y operación
3	Decisiones de arquitectura	5	Temas transversales
4	Dominio, actores y permisos	6	Infraestructura y despliegue
F0	Fundaciones	7	Calidad: pruebas y CI
F1	Identidad y cierre del área privada	8	Seguridad y cumplimiento
F2	Catálogo de predios	9	Plan, esfuerzo e hitos
F3	Proyecto y panel de obra	10	Riesgos y decisiones pendientes
F4	Documentos y fotos	A	Anexo: pantalla → endpoint
F5	Captación y notificaciones		

1 Resumen ejecutivo

El frontend está terminado y no tiene con qué hablar. La propuesta es construir el backend en siete fases, cada una cerrando un hueco concreto que hoy está pintado en la interfaz pero no opera, y en un orden que pone primero lo que hay que arreglar sí o sí: el área privada no está protegida.

La forma del backend no se inventa desde cero. El repositorio ya tiene un esqueleto — `app/api`, `app/core`, `app/services`, con un `gcs_service.py` entre ellos— y el frontend ya define las estructuras de datos que sabe pintar. La propuesta respeta las dos cosas: **FastAPI sobre PostgreSQL, desplegado en Google Cloud**, con el contrato de datos derivado de lo que las pantallas ya muestran.

LAS SIETE FASES DE UN VISTAZO

FASE	QUÉ ENTREGA	DURACIÓN	POR QUÉ VA EN ESE PUESTO
F0	Fundaciones: proyecto, base de datos, migraciones, CI, entorno local	2 semanas	Nada se puede construir dos veces bien si el suelo se mueve
F1	Identidad, sesiones y cierre real del área privada	2 semanas	URGENTE Hoy <code>/panel</code> y <code>/predios</code> son públicos
F2	Catálogo de predios, ficha, análisis de valor, filtros	3 semanas	Es la primera pantalla que un inversionista ve tras entrar
F3	Proyecto de obra y las diez vistas del panel	5 semanas	El núcleo del producto y la fase más grande
F4	Documentos y fotos sobre Cloud Storage	2 semanas	Depende de F3: los archivos cuelgan de un proyecto
F5	Solicitudes de acceso, diagnóstico, HUB, notificaciones	2 semanas	Es captación: no bloquea a nadie que ya sea cliente
F6	Operación del activo entregado: arriendo, bitácora, estado de cuenta	3 semanas	Sólo aplica cuando hay un activo entregado
F7	Observabilidad, respaldos, límites, cumplimiento	continuo	Empieza en F0 y se cierra formalmente al final

EL HITO QUE IMPORTA

Al terminar **F3** —unas doce semanas— existe una plataforma real: un inversionista entra con sus credenciales, ve su predio, entra a su proyecto y sigue su obra con datos verdaderos. F4 a F6 completan; F0 a F3 hacen que exista.

ADVERTENCIA DE ESFUERZO DEL LADO FRONTEND

El frontend tiene **dos árboles** para cada pantalla: el lienzo de 1920 px y la vista fluida de móvil. Cada integración se cablea dos veces. La estimación de cada fase incluye ese trabajo, pero conviene tenerlo presente al negociar plazos: no es duplicación accidental, es la arquitectura del frontend.

2 Punto de partida

2.1 Lo que existe

El frontend está en marcha: 21 rutas, cada una con vista de escritorio y vista fluida, con todo el contenido escrito dentro de los componentes. El backend es un esqueleto de carpetas con **doce archivos** `.py` a 0 bytes, más `schema.sql`, `seeds/data.sql`, `docker-compose.yml` y los dos `.env.example`, también vacíos.

2.2 Lo que el esqueleto ya decide

Aunque no haya código, los nombres de archivo son una decisión tomada y coherente, y esta propuesta la respeta en lugar de reabrir la:

ARCHIVO PREVISTO	LO QUE IMPLICA
<code>app/main.py</code> + <code>app/api/*</code>	Un único servicio HTTP con routers por dominio. Reparto típico de FastAPI
<code>app/core/database.py</code>	Base de datos relacional con capa propia de conexión y sesión
<code>app/core/security.py</code>	Autenticación y autorización propias, no delegadas a un proveedor externo
<code>app/services/gcs_service.py</code>	Google Cloud Storage , y por tanto GCP como nube. Es la señal más fuerte del esqueleto
<code>app/api/notificaciones.py</code>	Notificaciones como dominio propio, no un efecto colateral

2.3 Lo que el frontend ya define

Tres estructuras del frontend son, de hecho, el contrato mínimo que el backend tiene que alimentar. Están documentadas en `docs/api-reference.md`: el tipo **Predio** (catálogo y tarjeta), **Propiedad** (portafolio del inversionista, con una unión discriminada entre activo *en obra* y activo *arrendado*) y **PanelKey** (las diez vistas privadas).

UN DETALLE QUE VALE UNA DECISIÓN DE DISEÑO

Las cinco etiquetas de estado de la tarjeta de predio — **Disponible**, **Nueva oportunidad**, **Alta actividad**, **En proceso de reserva**, **Reservada**, más **Reserva liberada** — no son adornos: son **una máquina de estados de reserva ya diseñada**. El backend debe modelarla como tal, con transiciones explícitas, y no como un campo de texto libre.

3 Decisiones de arquitectura

3.1 La pila

PIEZA	PROPUESTA	POR QUÉ
Runtime / framework	Python 3.12 + FastAPI	Es lo que el esqueleto ya asume. Da OpenAPI gratis, y de ahí salen los tipos del frontend (§7.3)
Base de datos	PostgreSQL 16	El dominio es relacional y con integridad dura (dinero, aprobaciones). Además da JSONB para lo semiestructurado
ORM / migraciones	SQLAlchemy 2.0 + Alembic	Migraciones versionadas desde el primer día; <code>schema.sql</code> pasa a ser generado, no escrito a mano
Validación	Pydantic v2	Un solo sitio define la forma de entrada, salida y documentación
Archivos	Google Cloud Storage con URLs firmadas	Ya está decidido por <code>gcs_service.py</code> . Firmadas para no pasar bytes por la API
Cómputo	Cloud Run	Escala a cero, encaja con tráfico irregular de un portafolio privado, y evita administrar servidores
Trabajos en segundo plano	Cloud Tasks + un worker del mismo contenedor	Correo, miniaturas y recálculos no deben bloquear una petición. Evita montar Celery + Redis para poco volumen
Correo transaccional	Proveedor externo (SendGrid / Resend)	Credenciales de acceso, avisos de aprobación y confirmaciones. No merece infraestructura propia

LO QUE SE PROPONE NO HACER

- **Microservicios.** Un dominio, un equipo pequeño, un servicio. La separación por routers y servicios ya da orden suficiente.
- **GraphQL.** Las pantallas son fijas y conocidas; cada una tiene su endpoint. GraphQL resolvería un problema que aquí no existe y complicaría permisos.
- **Un CMS.** Ver §5.5: casi todo el texto del sitio está acoplado al diseño de 1920 px y debe quedarse en el código.
- **Proveedor de identidad externo** (Auth0, Firebase Auth). No hay registro público: las credenciales las emite Zequara. Un modelo propio es más simple y más barato, y `core/security.py` ya lo anticipa.

3.2 Forma de la API

REST con prefijo de versión `/api/v1`, JSON, verbos convencionales, y una regla de oro: **un endpoint por pantalla cuando la pantalla lo pide**. El panel tiene diez vistas y cada una necesita un agregado distinto; obligar al frontend a hacer seis llamadas y recomponer sería trasladarle trabajo del servidor.

```
GET /api/v1/predios?ciudad=bogota&capital=1000-2000&orden=score
GET /api/v1/predios/{slug}
GET /api/v1/predios/{slug}/analisis-valor
GET /api/v1/mis-propiedades
GET /api/v1/proyectos/{id}/resumen      ← una llamada por vista del panel
POST /api/v1/proyectos/{id}/aprobaciones/{aid}/decision
```

4 Dominio, actores y permisos

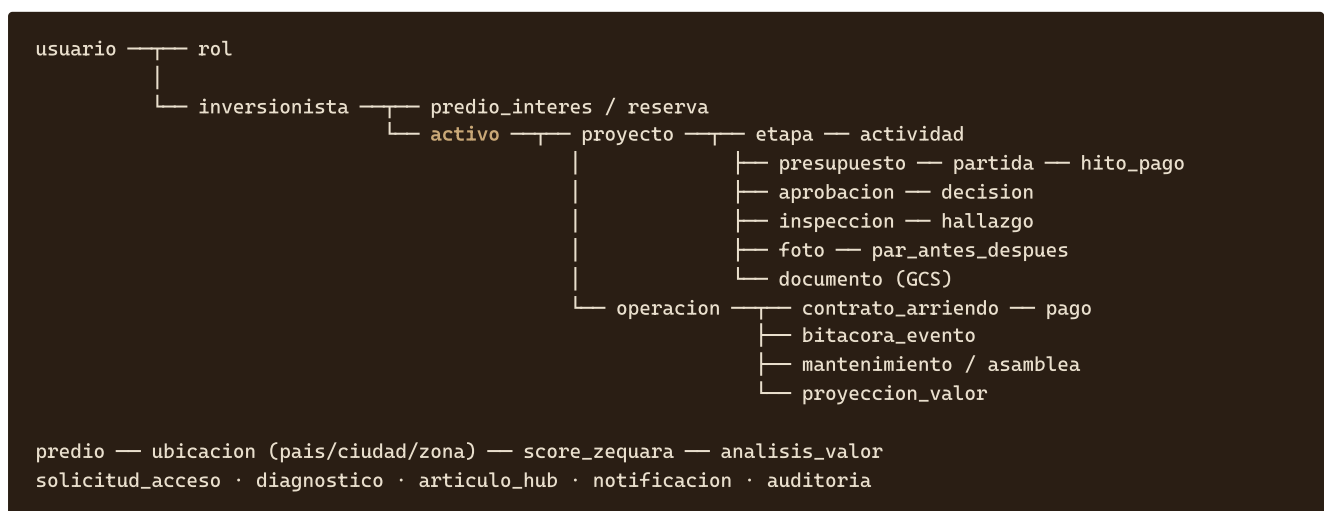
El modelo de permisos no es genérico: sale directamente de la promesa comercial. El sitio dice, con estas palabras, que la interventoría es «*un equipo independiente del que ejecuta la obra*» y que hay «*control real, no juez y parte*». Eso tiene que ser cierto **en la base de datos**, no sólo en la copia.

ROL	PUEDE	NO PUEDE
Inversionista	Ver su predio, su proyecto y sus documentos. Decidir aprobaciones. Escribir al gestor	Ver proyectos de otros. Modificar avance, presupuesto o hallazgos
Gestor de proyecto	Actualizar avance, subir fotos y documentos, responder mensajes, proponer aprobaciones	Crear ni editar hallazgos de interventoría , ni cerrar los suyos propios
Interventoría	Registrar inspecciones y hallazgos, verificar cierres	Modificar el avance que audita, ni el presupuesto
Analista / curador	Crear predios, cargar el análisis de valor, calcular el Score	Entrar a proyectos de obra
Administrador	Emitir credenciales, asignar gestores, ver la bitácora de auditoría	Borrar registros de auditoría (nadie puede)

REGLA DE ORO DEL MODELO

La separación gestor / interventoría se implementa como una restricción del sistema de permisos, comprobada en el servicio y cubierta por una prueba automatizada explícita. Si algún día alguien afloja esa regla, una prueba tiene que ponerse en rojo: es el argumento de venta del producto.

4.1 Mapa del dominio



FASE 0

Fundaciones: que el suelo no se mueva

2 semanas · sin dependencias · resultado: `docker compose up` levanta una API viva con migraciones

Objetivo

Que cualquiera clone el repositorio y tenga el backend corriendo en un comando, y que a partir de ahí todo cambio de esquema quede versionado. Es la fase que nadie ve y que decide si las seis siguientes cuestan lo estimado o el doble.

Alcance

- `requirements.txt` con versiones fijadas (hoy está vacío), y un `pyproject.toml` con las herramientas de estilo: `ruff`, `black`, `mypy`.
- `app/main.py`: fábrica de aplicación, montaje de routers, CORS restringido al dominio del frontend, manejadores de error globales.
- `app/core/config.py`: configuración con `pydantic-settings`, leída de variables de entorno. Se llenan a la vez los dos `.env.example` vacíos.
- `app/core/database.py`: motor, sesión por petición como dependencia de FastAPI, y *pool* dimensionado para Cloud Run.
- Alembic configurado, con la primera migración. `database/schema.sql` pasa a ser un volcado generado, no una fuente que se edita.
- `docker-compose.yml` (hoy vacío): Postgres + API con recarga en caliente + un visor de base de datos.
- Sobre transversal: `/health` y `/ready`, registro estructurado en JSON con identificador de correlación, y el sobre de error único que usará toda la API.
- CI en GitHub Actions: estilo, tipos y pruebas en cada *push*.
- Lado frontend: llenar `frontend/lib/api.ts` con un cliente tipado (`fetch` con base URL, manejo de error y credenciales) y `frontend/.env.local.example`.

EL SOBRE DE ERROR, DECIDIDO UNA VEZ

```
{ "error": { "code": "PREDIO_NO_ENCONTRADO",
            "message": "No existe un predio con ese identificador.",
            "details": {}, "request_id": "01HX..." } }
```

Un único formato para los cuatro casos que el frontend tendrá que pintar: no encontrado, sin permiso, validación fallida y error del servidor. Decidirlo aquí evita que cada pantalla invente el suyo.

CRITERIO DE ACEPTACIÓN

- `docker compose up` deja la API respondiendo y la base migrada, en una máquina limpia.
- `alembic upgrade head` y `alembic downgrade -1` funcionan las dos.
- CI verde con estilo, tipos y al menos una prueba de integración real contra Postgres.
- `/docs` sirve el OpenAPI, y de él ya se generan tipos TypeScript.

Identidad y cierre del área privada

2 semanas · depende de F0 · cierra el agujero de seguridad que hoy existe

EL PROBLEMA QUE RESUELVE

Hoy el *login* sólo comprueba que los campos no estén vacíos y hace `router.push("/predios")`. No hay sesión, ni *middleware*, ni comprobación en servidor: cualquiera que escriba `/panel/presupuesto` ve el panel. Mientras el contenido sea de muestra el daño es nulo; el día que haya un dato real de un cliente, deja de serlo. Por eso esta fase va antes que cualquier funcionalidad.

Alcance

MODELO

```

usuario      (id, email, hash_clave, nombre, iniciales, estado, ultimo_acceso,
              debe_cambiar_clave, creado_por)
rol          (inversionista | gestor | interventoria | analista | admin)
usuario_rol  (usuario_id, rol, ambito_id) ← el ámbito acota el rol a un activo
sesion       (id, usuario_id, emitida, expira, revocada, ip, agente)
auditoria    (id, actor_id, accion, entidad, entidad_id, antes, despues, cuando)

```

DECISIONES

- **Sin registro público.** La pantalla de acceso ya dice que las credenciales llegan por correo: el administrador crea el usuario, el sistema envía una clave temporal de un solo uso y fuerza el cambio en el primer ingreso.
- **Cookie de sesión** `HttpOnly`, `Secure`, `SameSite=Lax`, en lugar de un *token* en JavaScript. El frontend es Next.js con componentes de servidor: el *middleware* puede validar la cookie antes de renderizar, y así ninguna pantalla privada se pinta nunca sin permiso. Un *token* en `localStorage` obligaría a renderizar primero y comprobar después, y sería robable con un XSS.
- **Contraseñas con Argon2id.** Límite de intentos por cuenta y por IP, con espera creciente.
- **Auditoría desde el primer día.** La fase 3 tiene aprobaciones que mueven dinero; añadir la bitácora después es reescribir servicios.

ENDPOINTS

ENDPOINT	QUÉ HACE
POST <code>/api/v1/auth/login</code>	Valida credenciales, crea sesión, fija la cookie
POST <code>/api/v1/auth/logout</code>	Revoca la sesión
GET <code>/api/v1/auth/yo</code>	Usuario, roles y activos accesibles. Alimenta el avatar y la barra de proyecto
POST <code>/api/v1/auth/clave</code>	Cambio de clave, obligatorio en el primer ingreso
POST <code>/api/v1/auth/recuperar</code>	Envío de enlace de un solo uso, con caducidad

LADO FRONTEND

- `middleware.ts` protegiendo `/predios/*` y `/panel/*`.

- `LoginScreen` y `LoginCompact` : llamada real, estado de carga y de error (hoy no existen).
- El avatar `NR` y el nombre del proyecto, hoy fijos en `Shell` , pasan a venir de `/auth/yo` .

CRITERIO DE ACEPTACIÓN

- Sin sesión, `/panel` redirige a `/login` en el servidor, sin pintar nada.
- Un inversionista que pide el activo de otro recibe 404, no 403: no se confirma la existencia.
- Una prueba automatizada verifica que un gestor no puede escribir hallazgos de interventoría.
- Toda acción que cambia estado deja una fila en `auditoria` .

FASE 2

Catálogo de predios

3 semanas · depende de F1 · pantallas: `/predios`, `/predios/ficha`, `/predios/add-value`, `/predios/mis-propiedades`

Objetivo

Que las cuatro pantallas del área de predios dejen de leer `components/predios/data.ts` y lean la base de datos, con los seis filtros funcionando.

MODELO

```
ubicacion      (pais, ciudad, zona)
predio         (id, slug, ubicacion_id, titulo, especificaciones, area_m2,
               habitaciones, banos, parqueaderos, tipo_transformacion_id,
               precio_compra, precio_remodelacion, moneda, horizonte_anios,
               tir_estimada, estado_reserva, publicado_en)
tipo_transformacion (codigo, nombre, descripcion) ← el texto del tooltip, no un literal
score_zequara   (predio_id, valor, version_modelo, componentes JSONB, calculado_en)
predio_media    (predio_id, orden, gcs_path, tipo, texto_alternativo)
analisis_valor  (predio_id, inversion_total, arriendo_mensual, valor_esperado,
               roi_estimado, cascada JSONB, supuestos, vigente_hasta)
reserva         (predio_id, usuario_id, estado, creada, expira, liberada_en)
```

DOS DECISIONES QUE EVITAN DEUDA

- El Score Zequara se guarda con sus componentes y la versión del modelo. El tooltip promete «evaluación especializada de la zona y su afinidad con la estrategia Zequara»: guardar sólo el número 96 hace esa frase indefendible en cuanto alguien pregunte. Con componentes y versión, el número es explicable y recalculable.
- El estado de reserva es una máquina de estados con transiciones permitidas y caducidad automática, no un texto. Las seis etiquetas del diseño son sus estados; `Reserva liberada` y `Alta actividad` son estados derivados, calculados de la actividad reciente.

ENDPOINTS

ENDPOINT	NOTAS
GET <code>/api/v1/predios</code>	Filtros: país, ciudad, zona, capital, tipo de transformación, orden. Paginación por cursor
GET <code>/api/v1/predios/filtros</code>	Las opciones reales de cada desplegable, con conteos. Hoy los valores son literales
GET <code>/api/v1/predios/{slug}</code>	Ficha completa
GET <code>/api/v1/predios/{slug}/analisis-valor</code>	Alimenta Add Value
POST <code>/api/v1/predios/{slug}/reserva</code>	Activa el «Reservar ahora», que hoy es un botón muerto. Idempotente por clave de petición
GET <code>/api/v1/mis-propiedades</code>	Portafolio: resumen + un elemento por activo, con la métrica según esté en obra o arrendado

LADO FRONTEND

- `data.ts` se convierte en el tipo y en datos de prueba; los datos reales llegan del servidor.
- **El formateo se mueve.** Hoy la API tendría que devolver `"COP $3.100M"` porque el tipo `Predio` guarda el precio como texto ya compuesto. Se cambia a número + moneda, y el formateo pasa a una utilidad del frontend (§5.1).
- Se cablean los seis filtros, «Limpiar filtros» y el orden por Score.

CRITERIO DE ACEPTACIÓN

- Los ocho predios de muestra salen de la base y se pintan igual que hoy, en las dos vistas.
- Cada filtro reduce el conjunto, y el resultado se puede compartir por URL.
- Un predio ya reservado no admite una segunda reserva, ni con dos pulsaciones simultáneas.
- El listado responde por debajo de 300 ms con 10 000 predios cargados.

Proyecto y panel de obra

5 semanas · depende de F1 y F2 · seis de las diez vistas del panel

Objetivo

Que las vistas *Resumen*, *Avance*, *Presupuesto*, *Aprobaciones*, *Interventoría* y *Mi gestor* muestren el estado real de una obra, y que las aprobaciones se puedan decidir de verdad.

MODELO

```

activo      (id, predio_id, inversionista_id, estado, entregado_en)
proyecto    (id, activo_id, gestor_id, nombre, semana_actual, semanas_totales,
             estado, inicio, fin_previsto)
etapa       (proyecto_id, orden, nombre, estado, pct_avance, inicio, fin)
actividad   (etapa_id, nombre, estado, responsable, fecha_prevista)
nota_etapa  (etapa_id, autor_id, texto, creada)      ← hoy no persiste

presupuesto (proyecto_id, version, total_cerrado, total_ejecutado, moneda, vigente)
partida     (presupuesto_id, capitulo, nombre, cerrado, ejecutado)
hito_pago   (presupuesto_id, nombre, monto, vencimiento, estado, pagado_en)

aprobacion  (id, proyecto_id, titulo, descripcion, propuesta_por,
             impacto_costo, impacto_dias, impacto_interventoria,
             estado, vence, decidida_en, decidida_por, version)
decision    (aprobacion_id, actor_id, valor, motivo, cuando) ← inmutable

inspeccion  (proyecto_id, interventor_id, fecha, tipo, veredicto)
hallazgo    (inspeccion_id, titulo, descripcion, severidad, estado,
             aprobacion_id)    ← el enlace a Aprobaciones que ya insinúa el diseño

mensaje     (proyecto_id, autor_id, cuerpo, leído_en)
reunion     (proyecto_id, cuando, medio, enlace, estado)

```

APROBACIONES: EL CAMINO DE ESCRITURA MÁS DELICADO DEL SISTEMA

Es el único sitio donde el inversionista toma una decisión con consecuencias de dinero y de plazo. Requiere cuatro cosas que no se pueden añadir después:

- **Bloqueo optimista** por `version`: si la aprobación cambió desde que se cargó la pantalla, la decisión se rechaza y se pide recargar. Sin esto, dos pestañas abiertas pueden decidir sobre información distinta.
- **Clave de idempotencia** en la petición: una doble pulsación no puede producir dos decisiones.
- **Decisión inmutable**: no se actualiza, se inserta. Cambiar de opinión es una decisión nueva sobre una aprobación nueva.
- **Impacto guardado con la propuesta**, no calculado al decidir: lo que el inversionista aceptó es lo que vio.

ENDPOINTS

ENDPOINT	ALIMENTA
GET /proyectos/{id}/resumen	/panel
GET /proyectos/{id}/avance	/panel/avance
POST /proyectos/{id}/etapas/{eid}/notas	Las notas por etapa, que hoy se pierden
GET /proyectos/{id}/presupuesto	/panel/presupuesto . El sobrecosto se calcula, nunca se guarda como texto
GET /proyectos/{id}/aprobaciones	/panel/aprobaciones y el badge del sidebar (hoy un 2 fijo)
POST /proyectos/{id}/aprobaciones/{aid}/decision	Los botones «Aprobar» y «Rechazar», hoy sin manejador
GET /proyectos/{id}/interventoria	/panel/interventoria
GET /proyectos/{id}/gestor · POST ../mensajes	/panel/gestor

CRITERIO DE ACEPTACIÓN

- Las seis vistas pintan datos de la base, en escritorio y en móvil.
- Una aprobación decidida se refleja en el historial, en el badge y en la auditoría.
- Dos decisiones simultáneas sobre la misma aprobación: una gana, la otra recibe conflicto.
- Un gestor autenticado recibe 403 al intentar escribir un hallazgo de interventoría.
- El sobrecosto que se muestra siempre iguala **ejecutado - cerrado** sumado por partidas.

FASE 4

Documentos y fotos

2 semanas · depende de F3 · pantallas: `/panel/documentos`, `/panel/fotos` y las descargas del Resumen

Objetivo

Que el expediente del proyecto sea real: contrato, cifras, cronograma, planos y estudios descargables, y la galería de obra con su comparador antes/después.

MODELO

```
documento (id, proyecto_id, tipo, nombre, version, estado, gcs_path,
           bytes, sha256, mime, visible_para[], subido_por, subido_en)
foto       (id, proyecto_id, etapa_id, ambiente, tomada_en, gcs_path,
           derivadas JSONB, subida_por)
foto_par   (proyecto_id, ambiente, foto_antes_id, foto_despues_id)
```

FOTO_PAR : LO QUE EL COMPARADOR NECESITA Y HOY NO TIENE

Los dos comparadores del frontend —el arrastrable de escritorio y el táctil de móvil— no muestran una foto: muestran **una pareja** del mismo ambiente en dos momentos. Modelar la pareja explícitamente evita adivinarla por nombre de archivo, que es exactamente lo que hoy hace la vista táctil con el patrón `{slug}-antes.webp / {slug}-despues.webp`.

DECISIONES

- **URLs firmadas en las dos direcciones.** Para subir, la API entrega una URL de escritura de vida corta y el navegador sube directo a GCS. Para descargar, una URL de lectura de pocos minutos, emitida sólo tras comprobar permiso. La API nunca mueve los bytes: es lo que hace que Cloud Run siga siendo barato.
- **Derivadas al subir**, en segundo plano: miniatura, versión de galería y versión completa, en AVIF y WebP. El frontend ya sirve las imágenes con `next/image` y un `sizes` exacto; conviene alimentarlo con tamaños coherentes.
- **Verificaciones de carga:** lista blanca de tipos, tamaño máximo, análisis antivírico y `sha256` guardado para detectar duplicados y corrupción.
- **Visibilidad por rol en cada documento.** No todo el expediente es para el inversionista; algunos anexos son internos.

CRITERIO DE ACEPTACIÓN

- Una descarga funciona para su dueño y falla para un tercero, incluso con la URL copiada tras caducar.
- Subir una foto de 12 MP desde el móvil del gestor termina y genera derivadas sin bloquear la petición.
- El comparador carga una pareja real de un ambiente concreto.
- Ningún archivo se sirve a través de la API.

FASE 5

Captación y notificaciones

2 semanas · depende de F0 (puede adelantarse) · pantallas: `/solicitud-acceso`, diagnóstico, `/hub`

Objetivo

Que lo que hoy se pierde se guarde: la solicitud de acceso, las siete respuestas del Diagnóstico Patrimonial y las suscripciones del HUB. Y que la campana del panel deje de ser un adorno.

MODELO

```
solicitud_acceso (id, nombre, email, telefono, mercados[], capital_rango,
                 mensaje, origen, estado, creada, atendida_por)
diagnostico      (id, respuestas JSONB, perfil_calculado, nombre, email,
                 telefono, completado, creada)
suscripcion      (email, confirmada_en, origen, baja_en)
articulo_hub     (slug, tipo, titulo, resumen, cuerpo, categorias[],
                 portada_gcs, publicado_en, destacado)
notificacion     (usuario_id, tipo, entidad, entidad_id, titulo,
                 leida_en, creada)
```

ESTOS SON LOS ÚNICOS ENDPOINTS PÚBLICOS DE ESCRITURA

Y por eso el único blanco de abuso del sistema. Necesitan, desde el primer despliegue: límite de tasa por IP y por correo, prueba anti-robot en los formularios, honeypot, validación estricta, y **ninguna respuesta que revele si un correo ya existe**. El diagnóstico además guarda parcialmente —el diseño ya tiene un paso de corte antes de pedir los datos—, así que hay que decidir cuánto se conserva de un recorrido abandonado (ver §8.2).

NOTIFICACIONES

Un único servicio con dos salidas: la campana del panel (en base de datos) y el correo transaccional (en cola). Los disparadores nacen del dominio ya modelado: aprobación pendiente, hallazgo abierto, hito de pago próximo, documento nuevo, mensaje del gestor. El badge `2` del sidebar pasa a ser el conteo real.

CRITERIO DE ACEPTACIÓN

- Una solicitud enviada queda registrada y notifica al equipo comercial.
- Un diagnóstico completo guarda las siete respuestas y el perfil calculado.
- Cien envíos automatizados desde una IP se frenan sin tumbar el servicio.
- El HUB pinta artículos de la base y sus filtros funcionan.

FASE 6

Operación del activo entregado

3 semanas · depende de F3 y F4 · pantallas: `/panel/operacion` `/panel/valor`

Objetivo

Cubrir la vida del inmueble después de la entrega: arrendado, rentando y administrado. Es la vista más grande del panel (2299 px de lienzo) y la única que entra con su propio estado, «*Entregado y en operación · Arrendado*».

MODELO

```
contrato_arriendo (activo_id, inquilino_id, canon, moneda, inicio, fin,
                  incremento_anual, deposito, estado)
inquilino         (nombre, documento, contacto, verificado_en)
pago              (contrato_id, periodo, monto, vencimiento, pagado_en,
                  medio, estado)
estado_cuenta     (activo_id, periodo, ingresos, gastos, retenciones, neto)
bitacora_evento   (activo_id, tipo, titulo, detalle, ocurrido_en, adjuntos[])
mantenimiento     (activo_id, tipo, proveedor, costo, estado, programado, cerrado)
asamblea          (activo_id, fecha, tema, acta_documento_id, cuota_extra)
proyeccion_valor  (activo_id, escenario, valor, tir, supuestos JSONB,
                  calculada_en, vigente_hasta)
```

DECISIONES

- **La bitácora es un registro de eventos tipado**, con tipos cerrados (pago, mantenimiento, asamblea, visita, incidencia, documento). Los filtros de la vista, hoy decorativos, se convierten en filtros por tipo y por rango de fechas sobre un índice.
- **El estado de cuenta se calcula por periodo** y se guarda cerrado. Un histórico que cambia solo porque cambió una regla de cálculo es un problema contable, no un detalle técnico.
- **La proyección de valor guarda sus supuestos y su caducidad**. La vista lleva un descargo de responsabilidad en el diseño; los supuestos guardados son lo que lo respalda.

CRITERIO DE ACEPTACIÓN

- Los filtros de la bitácora filtran de verdad y la vista responde por debajo de 400 ms con 5 000 eventos.
- Un estado de cuenta cerrado no cambia al recalcular periodos posteriores.
- Un pago registrado aparece en bitácora, estado de cuenta y notificación, sin duplicarse.

Endurecimiento y operación

empieza en F0, se cierra formalmente al final · sin ella el sistema funciona hasta el día que falla

Alcance

ÁREA	QUÉ SE ENTREGA
Observabilidad	Registro estructurado con identificador de correlación, métricas de latencia y error por endpoint, trazas, y alertas sobre tasa de error y saturación de conexiones
Respaldos	Copias automáticas de Postgres con recuperación a un punto en el tiempo, y un ensayo de restauración documentado . Un respaldo sin ensayo no es un respaldo
Rendimiento	Revisión de índices contra las consultas reales, caché de las respuestas públicas del catálogo, presupuesto de latencia por endpoint
Límites	Límite de tasa por IP, por usuario y por endpoint sensible; cuotas de subida
Seguridad	Análisis de dependencias en CI, cabeceras de seguridad, rotación de secretos, revisión externa antes del primer cliente real
Cumplimiento	Ley 1581 de 2012: aviso de privacidad, registro de consentimiento, atención de derechos, retención y borrado (ver §8.2)
Operación	Guías de actuación: cómo restaurar, cómo revocar sesiones, cómo emitir credenciales, a quién llamar

5 Temas transversales

5.1 Dinero

Nunca en coma flotante. Enteros en la unidad mínima más un código de moneda ISO. Hoy el frontend recibe `"COP $3.100M"` ya compuesto; la API debe devolver `{ "monto": 3100000000, "moneda": "COP" }` y el formateo pasa a una utilidad del frontend, compartida por las dos vistas. El sitio maneja Colombia y Panamá, así que la moneda es un dato, no una constante.

5.2 Fechas y zona horaria

Almacenar siempre en UTC con zona explícita; presentar en `America/Bogota`. Los vencimientos de aprobaciones y de hitos de pago dependen del día local: una diferencia de cinco horas cambia si algo venció.

5.3 Escrituras seguras

- **Clave de idempotencia** obligatoria en toda escritura que produce un efecto: reserva, decisión de aprobación, subida, pago.
- **Bloqueo optimista** por versión donde hay concurrencia real.
- **Paginación por cursor**, no por número de página: los listados se ordenan por Score y por fecha, y con desplazamiento se saltan o repiten filas.

5.4 Contratos tipados entre frontend y backend

FastAPI publica OpenAPI. En CI se generan los tipos TypeScript (`openapi-typescript`) y se escriben en `frontend/lib/api-types.ts`. Si el backend cambia una respuesta y el frontend no se adapta, **la compilación falla**. Esto importa más aquí que en un proyecto normal: hay dos árboles que consumen los mismos datos, y un cambio silencioso rompería sólo la mitad de las pantallas —probablemente la que nadie mira en el móvil.

5.5 Contenido: lo que *no* debe ir al backend

Es la trampa más clásica de este proyecto. Casi todo el texto del sitio está escrito dentro de componentes posicionados al píxel: los titulares de la home van partidos en dos pesos tipográficos, la comparativa tiene siete pasos con negritas dentro de las frases, el diagnóstico tiene siete preguntas con cuatro opciones cada una. Sacar eso a un CMS costaría semanas y produciría un sitio peor.

La regla propuesta:

- **Se queda en el código** el texto de marca acoplado al diseño: home, Cómo operamos, Oportunidades, los dos modales, todos los rótulos y microcopias.
- **Va al backend** lo que es dato del negocio y cambia sin diseñador: predios, proyectos, presupuestos, documentos, fotos, artículos del HUB, contratos.

La frontera es sencilla de aplicar: si cambiarlo exige mover un píxel, es diseño; si cambia porque cambió el mundo, es dato.

6 Infraestructura y despliegue

```
Internet
├── Vercel / Cloud Run — frontend Next.js
│   └── (fetch con cookie de sesión)
├── Cloud Load Balancing — Cloud Run: API FastAPI
│   ├── Cloud SQL (PostgreSQL 16, IP privada)
│   ├── Cloud Storage (documentos y fotos)
│   ├── Secret Manager (claves y credenciales)
│   ├── Cloud Tasks — worker (correo, derivadas)
│   └── Cloud Logging / Monitoring
```

DECISIÓN	MOTIVO
Dos entornos: <code>staging</code> y <code>producción</code>	Con bases y buckets separados. Las migraciones se prueban en staging con un volcado anonimizado
Cloud SQL con IP privada	La base nunca se expone a Internet; el acceso va por conector desde Cloud Run
Migraciones como paso de despliegue	<code>alembic upgrade head</code> antes de cambiar el tráfico, y sólo migraciones compatibles hacia atrás para poder revertir
Imagen en Artifact Registry, etiquetada por <code>commit</code>	Revertir es apuntar al contenedor anterior, no reconstruir
Región <code>southamerica-east1</code> o <code>us-east1</code>	Latencia hacia Colombia y Panamá. Decidir junto con dónde se aloje el frontend

7 Calidad: pruebas y CI

NIVEL	HERRAMIENTA	QUÉ CUBRE
Unitario	<code>pytest</code>	Cálculos: sobrecosto, Score, ROI, estado de cuenta, transiciones de reserva
Integración	<code>pytest</code> + Postgres real en contenedor	Cada endpoint contra base de datos de verdad. Sin bases en memoria: el dialecto miente
Permisos	Batería propia por rol	Cada endpoint probado con los cinco roles. Aquí vive la prueba de gestor $\neq$ interventoría
Contrato	<code>openapi-typescript</code> en CI	Los tipos del frontend se regeneran; si divergen, falla la compilación
Carga	<code>k6</code> , antes de F7	Listado de predios, resumen del panel y bitácora, que son las consultas caras

Cobertura como umbral mínimo en los servicios de dominio, no como número global. Lo que hay que tener cubierto al 100 % es el camino de aprobaciones y el de permisos.

8 Seguridad y cumplimiento

8.1 Seguridad

- Cerrar el área privada es la fase 1, no una tarea posterior.
- Autorización comprobada **en el servicio**, nunca sólo en el router, y por pertenencia del recurso: un identificador válido de otro activo devuelve 404.
- Argon2id, límite de intentos, sesiones revocables, caducidad y renovación.
- Consultas siempre parametrizadas; nada de SQL compuesto por concatenación.
- Secretos en Secret Manager, nunca en el repositorio. Los `.env.example` llevan nombres de variable, jamás valores.
- CORS restringido a los dominios propios; cookies `HttpOnly`, `Secure`, `SameSite`.
- Auditoría inmutable de toda acción que cambia estado.

8.2 Datos personales (Colombia, Ley 1581 de 2012)

El sistema captura datos personales de residentes en Colombia —solicitudes de acceso, diagnóstico patrimonial con rango de capital, inquilinos con documento de identidad—, así que el régimen de habeas data aplica de lleno. Lo que hay que tener resuelto antes del primer formulario en producción:

- **Autorización previa, expresa e informada**, con registro de cuándo y con qué texto se dio. Un *checkbox* sin guardar la versión del aviso no sirve como prueba.
- **Finalidad declarada** y aviso de privacidad accesible.
- **Derechos del titular**: consulta, actualización, rectificación y supresión, con un procedimiento y un plazo.
- **Retención con caducidad**: un diagnóstico abandonado sin datos de contacto no debería conservarse indefinidamente.
- **Cifrado en reposo y en tránsito**, y minimización: no pedir lo que no se va a usar.

Conviene revisarlo con asesoría legal; aquí se señala como requisito de diseño, no como dictamen jurídico.

9 Plan, esfuerzo e hitos

Estimación con un **desarrollador de backend a tiempo completo** más **disponibilidad parcial de frontend** para cablear cada fase en sus dos árboles. Son estimaciones, no compromisos: el margen razonable es $\pm 30\%$, y la fase 3 es la de mayor incertidumbre.

FASE	ENTREGA	BACKEND	FRONTEND	ACUMULADO
F0	Fundaciones	2 sem	0,3 sem	2 sem
F1	Identidad y cierre del área privada	2 sem	0,7 sem	4 sem
F2	Catálogo de predios	3 sem	1,5 sem	7 sem
F3	Proyecto y panel de obra	5 sem	2,5 sem	12 sem ← plataforma usable
F4	Documentos y fotos	2 sem	1 sem	14 sem
F5	Captación y notificaciones	2 sem	1 sem	16 sem
F6	Operación del activo	3 sem	1,5 sem	19 sem
F7	Endurecimiento	continuo	—	~20 sem

SI HAY QUE RECORTAR

- **Mínimo defendible**: F0 + F1. Cierra el agujero de seguridad y deja base para todo lo demás. Cuatro semanas.
- **Demostrable a un inversionista real**: F0 → F3. Doce semanas.
- **F5 se puede adelantar** y correr en paralelo si captar contactos es la prioridad comercial: sólo depende de F0.
- **F6 se puede posponer** hasta que exista el primer activo entregado. Antes de eso, no tiene usuarios.

10 Riesgos y decisiones pendientes

RIESGO	IMPACTO	MITIGACIÓN
El área privada sigue abierta mientras se construye otra cosa	ALTO	F1 va segunda, antes de cualquier funcionalidad. No negociable
Cada integración se cablea dos veces por los dos árboles del frontend	MEDIO	Extraer los datos a un módulo compartido por vista, como ya hace <code>predios/data.ts</code>
El contenido está dentro de los componentes	MEDIO	La frontera diseño/dato de \$5.5, aplicada fase por fase y no de golpe
Los tipos del frontend guardan importes ya formateados	MEDIO	Cambiar a número + moneda en F2, cuando sólo hay una pantalla que lo usa
La fase 3 es grande y tiene el camino de escritura más delicado	MEDIO	Partirla en dos entregas: lectura de las seis vistas primero, decisión de aprobaciones después
Falta un dueño del dato de obra	BAJO	Definir en F3 quién actualiza el avance y con qué frecuencia. Sin eso, el panel muestra datos viejos y pierde credibilidad

DECISIONES QUE HAY QUE TOMAR ANTES DE EMPEZAR

1. **¿Se confirma GCP?** El esqueleto lo asume por `gcs_service.py`. Si el frontend se despliega en Vercel, conviene decidir si la API va también allí o se queda en Cloud Run junto a la base.
2. **¿Quién carga el avance de obra?** El gestor desde el móvil en sitio, o un administrador desde el escritorio. Cambia las prioridades de F3 y F4.
3. **¿Hay integración contable?** Si el estado de cuenta debe conciliar con un sistema contable existente, eso es alcance adicional en F6.
4. **¿Cuántos activos por inversionista?** El modelo los soporta varios, pero define si el panel necesita un selector de proyecto en la barra superior.
5. **¿Multi-moneda desde el principio?** El catálogo ya muestra Panamá. Decidirlo en F2 es baratísimo; decidirlo en F6 no.

A Anexo: pantalla → endpoint → fase

Trazabilidad completa entre lo que hoy está pintado y no opera, y la fase que lo resuelve.

PANTALLA	QUÉ NO FUNCIONA HOY	ENDPOINT QUE LO RESUELVE	FASE
/login	No autentica; entra directo	POST /auth/login	F1
Todo /panel/*, /predios/*	Accesibles sin sesión	middleware + GET /auth/yo	F1
/predios	Los seis filtros no filtran	GET /predios, GET /predios/filtros	F2
/predios/ficha	«Reservar ahora» no hace nada	POST /predios/{slug}/reserva	F2
/predios/add-value	Cifras fijas	GET /predios/{slug}/analisis-valor	F2
/predios/mis-propiedades	Portafolio fijo	GET /mis-propiedades	F2
/panel	Todo fijo	GET /proyectos/{id}/resumen	F3
/panel/avance	Las notas por etapa no persisten	POST /proyectos/{id}/etapas/{eid}/notas	F3
/panel/presupuesto	Cifras fijas	GET /proyectos/{id}/presupuesto	F3
/panel/aprobaciones	Aprobar/rechazar sin manejador; badge 2 fijo	POST .../aprobaciones/{aid}/decision	F3
/panel/interventoria	Registro fijo	GET /proyectos/{id}/interventoria	F3
/panel/gestor	Escribir, agendar y calendario	POST .../mensajes, POST .../reuniones	F3
/panel/documentos	Descargas sin archivo detrás	GET /documentos/{id}/url	F4
/panel/fotos	Galería y comparador fijos	GET /proyectos/{id}/fotos	F4
/solicitud-acceso	No envía el formulario	POST /solicitudes-acceso	F5
Diagnóstico Patrimonial	No envía respuestas ni contacto	POST /diagnosticos	F5
/hub	Buscador, pestañas y chips no filtran	GET /articulos	F5
Campana del panel	Adorno	GET /notificaciones	F5
/panel/operacion	Los filtros de la bitácora no filtran	GET /activos/{id}/bitacora	F6
/panel/valor	Proyección fija	GET /activos/{id}/proyeccion-valor	F6

REFERENCIAS EN EL REPOSITORIO

docs/arquitectura.md — cómo está construido el frontend.

docs/flujo-negocio.md — el recorrido pantalla por pantalla.

docs/api-reference.md — inventario de puntos de integración, con archivo y línea.